

Examen de rattrapage
Master 1 SAR
UE MU4IN401

L. Arantes, P. Sens, J. Sopena,

Juin 2021

2 heures – **Tout document papier autorisé**
Tout appareil électronique interdit
Le barème est donné à titre indicatif

Exercice 1 – Questions de cours (2,5 points)

On considère l'UNIX du TD. Toutes les réponses doivent être justifiées.

Question 1 $\frac{1}{2}$ point

Est-il possible qu'un processus traite une seule fois un signal qui lui a été envoyé deux fois (on suppose que le signal n'est pas masqué, ni ignoré et que le handler est bien défini) ?

Question 2 $\frac{1}{2}$ point

Dans quels cas un processus à l'état SRUN peut rester non chargé en mémoire ?

Question 3 $\frac{1}{2}$ point

L'inode d'un même fichier peut-elle être présente à un même moment à plusieurs emplacements dans la table inode en mémoire ?

Question 4 1 point

On considère un processus P1 endormi. Sa priorité est égale à 110. A $t=10s$, le processus P2 de priorité 120 est élu et lance un appel système qui dure 0.3s. Cet appel réveille le processus P1 via un wakeup au bout de 0.1s. A quel moment au plus tôt, P1 s'exécutera ?

Exercice 2 – Système de fichiers (3,5 points)

On considère le fichier "fich1" qui contient la chaîne de caractère "rattrapage2021" et est désigné par l'inode 20. Le fichier "fich2" est un lien *hard* vers le fichier "fich1". Les inodes ont la même structure que celle vue en TD. Un programme fait les instructions suivantes :

```

1 int main() {
2   int fd1, fd2, fd3;
3   char buf[4];
4   fd1 = open("./fich1", O_RDWR);
5   fd2 = open("./fich2", O_RDONLY);
6   if (fork() == 0) {
7     if (fork() == 0) {
8       write(fd1, "XXX", 3);
9       read(fd1, buf, 3);
10      buf[3] = 0;
11      printf("%s\n", buf);
12      exit(0);
13    }
14    else {
15      fd3 = open("./fich1", O_WRONLY);
16      close(fd1);
17      close(fd2);
18      wait(NULL);
19      exit(0);
20    }
21    wait(NULL);
22    return 0;
23  }

```

pas APPEND

fich 1 = XXX
buf = XXX
print XXX

Question 1 1 point

Quel est l'affichage fait par ce programme ? Quel est le contenu du fichier "fich1" et "fich2" à la fin de l'exécution ?

Question 2 2½ points

Donnez l'état, juste après la ligne 9 et en supposant que la ligne 17 a pu être exécutée. Vous indiquerez les tables de descripteurs de fichiers (`u_ofile`), des fichiers ouverts (`file`), et des inodes (`inode`). Faites un schéma en pensant à représenter les champs `f_count`, `f_offset`, `f_flags`, `i_count`, `i_number` et `i_size`. Pour les inodes vous représenterez juste l'état de l'inode 20.

Exercice 3 – Buffer cache (5 points)

Nous considérons un buffer cache avec une capacité de **4 buffers**. Initialement la `bfreelist` est la suivante (de la tête vers la queue), en considérant le format `(dev, bloc, [D])`.

`(0, 5, -)`, `(0, 9, D)`, `(0, 11, -)`, `(1, 1, -)`

Un `D` indique que le buffer a le flag `DELWRI` positionné. Initialement, il n'y a aucun buffer en dehors de la `bfreelist`.

Question 1 1 point

Faites un schéma de la structure du buffer cache en considérant une table `hbuf` de taille 5 et la fonction de hachage `fh(dev, bloc) = (dev + bloc) % 5`.

Question 2 2 points

Le noyau exécute la séquence suivante :

```

1 struct buf *bp;
2 bp = bdwrite(0, 5);
3 brelse(bp);
4 bp = bread(0, 9);
5 brelse(bp);
6 bp = bread(1, 2);
7 brelse(bp);

```


Donnez l'état de la bfreelist (la liste des blocs qui la composent de la tête vers la queue) après les lignes 3, 5 et 7 en précisant les flags DELWRI ainsi que le nombre d'entrées/sorties physiques générées par cette séquence.

Question 3 2 points

Reprendre la question précédente mais en considérant dans la configuration initiale de la bfreelist que le buffer $\langle 0, 11 \rangle$ a le flag DELWRI positionné.

L'état initial de la bfreelist est alors : $\langle 0, 5, - \rangle$, $\langle 0, 9, D \rangle$, $\langle 0, 11, D \rangle$, $\langle 1, 1, - \rangle$.

On fait l'hypothèse que l'écriture sur disque d'un bloc est plus lente que la lecture d'un bloc.

Exercice 4 – Appel système - Fichier (5 points)

On souhaite implémenter dans le noyau du TD l'appel système suivant.

- `void rewind(int fd)`. Cet appel positionne l'offset à 0 pour tous les processus manipulant le même fichier (inode) que celui référencé par *fd*, *fd* étant le descripteur de fichier retourné par `open`.

Question 1 2½ points

Donnez le code dans le noyau de l'appel système `sys_rewind()` correspondant à `void rewind(int fd)`.

Question 2 2½ points

Modifiez la fonction pour que les processus manipulant le fichier en écriture aient l'offset positionné à la fin du fichier, les autres (i.e., les processus n'ayant pas le mode d'écriture sur le fichier) ont eux l'offset mis à 0. De plus l'appel système doit retourner le nombre processus manipulant le fichier en écriture.

Exercice 5 – Synchronisation (4 points)

On souhaite implémenter dans le noyau le mécanisme de partage suivant. Une variable globale *Res* a deux champs : un entier *value* qui contient une valeur et un entier *count* qui compte le nombre de mises à jour de *Res.value*. *Res* est définie dans le noyau de la manière suivante :

```
struct part {  
    int count = 0;  
    int value;  
} Res;
```

La fonction `void update_res(int v)` permet de mettre à jour le champs *value* à la valeur *v*. La fonction `int consomme_res()` retourne la valeur courante.

Question 1 1½ points

Donnez le code dans le noyau des appels système `sys_update_res()` et `sys_consume_res()` correspondant à `update_res(int v)` et `int consomme_res()`.

Question 2 2½ points

On souhaite modifier les fonctions de la manière suivante.

En appelant `int consomme_res(int n)`, un processus sera **bloqué** jusqu'à ce qu'**au moins** *n* **nouveaux appels** à `update_res(int v)` aient été effectués depuis son appel à `consomme_res`. Le processus appelant s'endormira alors à la priorité *PRES*.

Par exemple si un processus fait l'instruction `int a = consomme_res(10)`, il récupérera dans *a* la valeur de *Res.value* qu'après au moins 10 nouveaux appels à `update_res(int v)`. Donnez le nouveau code des appels système `sys_update_res()` et `sys_consume_res()`.

Annexe

```
1 #define NADDR 13
2
3 struct inode
4 {
5     char        i_flag;
6     char        i_count; /* reference count */
7     dev_t       i_dev;   /* device where inode resides */
8     ino_t       i_number; /* i number, 1-to-1 with device address */
9     unsigned short i_mode;
10    short       i_nlink; /* directory entries */
11    short       i_uid;   /* owner */
12    short       i_gid;   /* group of owner */
13    off_t       i_size;  /* size of file */
14    struct {
15        daddr_t   i_addr[NADDR]; /* if normal file/directory */
16        daddr_t   i_lastr;       /* last logical block read (for read-
17                                ahead) */
18    };
19 };
20
21 struct inode inode[NINODE]; /* The inode table itself */
22
23 /*
24  * One file structure is allocated
25  * for each open/creat/pipe call.
26  * Main use is to hold the read/write
27  * pointer associated with each open
28  * file.
29  */
30 struct file
31 {
32     char        f_flag;
33     cnt_t       f_count; /* reference count */
34     int         *f_inode; /* pointer to inode structure */
35     long        f_offset; /* read/write character index */
36 } file[NFILE];
37
38 /* open parameters */
39 #define O_RDONLY    0
40 #define O_WRONLY    1
41 #define O_RDWR      2
```